Fall 2026 DRP: Formal Languages

Michael Lee

I plan to keep these updated as the semester goes along, to serve as a record of the material and problems we've covered. Let me know if you spot any typos. These draw heavily on the lecture notes by Professor Timothy Ng, but all errors are mine alone.

# 1 Introduction: Theory of Computation (Sep 9th)

This is a project on computer science as a branch of mathematics. Math is the study of different kinds of structures, for example

- number theory is about the structure of the integers and their divisibility.
- combinatorics is about the enumeration of discrete structures
- calculus is about the properties of continuous structures, which interact with differentiation and integration operators.

What do we study in computer science? At the risk of sounding meta, we study the structures of *problems*, and the *algorithms* which solve them. But what does it mean to study the structure of problems?

In an intro programming course like CS 312, we usually think of problems as functions from some input to some output e.g. we compute the factorial function. In this setting, however, we will separate the definition of a function from the algorithm that computes it.

**Definition 1.1.** A function $f : A \to B$ is a binary relation i.e. $f \subset A \times B$ such that for every $a \in A$, there exists exactly one $b \in B$ such that $(a, b) \in f$ (we usually write this as $f(a) = b$.)

Two important observations:

- There's nothing in this definition about the algorithm on how to produce the required $b$ given $a$; we only know that it exists.
- A function is itself a kind of set. Indeed, $f \subset A \times B$. Therefore, we will interchange between functions and sets based on whichever is easier to work with.

Similarly, given a definition of set, there might be sometimes no algorithm to tell us whether an element is a member of the set or not. For example, one very famous such set, called Fermat's last theorem, is the following

$$\{(a, b, c) \in \mathbb{Z}^3 : \exists n \geq 3, a^n + b^n = c^n\}$$

In this DRP, we will try to study what kinds of sets admit an algorithm that can check if a given object is an element of the said set.

**Remark 1.2.** *In computer science, problems are sets of strings.*

Remember that depending on your preferred programming languages, functions can take arrays, lists, dictionaries, even nested arrays (i.e. arrays of arrays), sets, floating points, memory pointers, etc. These can all be encoded by strings. To simplify our problem, we study the setting of string functions with binary outputs

$$P : \text{Strings} \to \{\text{Yes}, \text{No}\}$$

which give rise to an equivalent set

$$\{x \in \text{Strings} : P(x) = \text{Yes}\}.$$

So, we can phrase our question as:

**Question 1.3.** *How do we determine whether a given string lies in some defined set?*

**Definition 1.4.** Sets of strings are called languages.

This definition is not surprising considering the connections we will see to linguistics.

If all of this sounds terribly abstract, don't worry because we will revisit these concepts in due time after many examples. In a nutshell, any computer process consists at its core of just elaborate sequences of 1s and 0s. We want to understand the capabilities and limits of these bit operations.

Let's start working with some concrete strings.

# 2 Mathematical Induction

(adapted from MATH 161 UChicago Script 0) First, a detour into induction. We let $\mathbb{N}$ denote the natural numbers. In set notation,

$$\mathbb{N} = \{0, 1, 2, \dots, n, \dots\}.$$

Induction is a way to prove that a statement holds for all the natural numbers.

**Theorem 2.1.** *For each natural numbers $n$, let $P(n)$ be a proposition. Suppose we show the following two results:*

*(a) $P(c)$ is true.*

*(b) For each natural number $k$, if $P(k)$ is true, then $P(k+1)$ is also true.*

*Then $P(n)$ is true for all natural numbers $n \geq c$.*

We call part (a) the base case and part (b) the inductive step. The assumption that $P(k)$ is true is called the inductive hypothesis.

**Example 2.2.** *Let's use induction to prove that for all $n \geq 1$,*

$$\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2}.$$

Here's a video going over this example: `https://www.youtube.com/watch?v=wblW_M_HVQ8`.

*Proof.* For this problem, $P(n)$ denotes the property that

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

is true. First, we check $P(c)$ for the base case i.e. $c = 1$. We have

$$\sum_{i=1}^{1} i = 1 = \frac{1(1+1)}{2}.$$

Second, we carry out the inductive step. Suppose that we have the inductive hypothesis $P(k)$:

$$\sum_{i=1}^{k} i = \frac{k(k+1)}{2}.$$

Then

$$\begin{aligned}
\sum_{i=1}^{k+1} i &= k + 1 + \sum_{i=1}^{k} i \\
&= k + 1 + \frac{k(k+1)}{2} \\
&= (k+1)\left(1 + \frac{k}{2}\right) \\
&= (k+1)\frac{k+2}{2} \\
&= \frac{(k+1)(k+2)}{2},
\end{aligned}$$

which confirms that $P(k+1)$ would also be true. Therefore, the statement holds for all $n \geq 1$. □

**Example 2.3.** *We can use induction prove different kinds of statements. For instance, we can use induction to show that $n \leq 2^n$ for all $n \geq 1$.*

*Proof.* For the base case, we check $1 \leq 2^1$. For the inductive step, suppose that we have $k \leq 2^k$. Then

$$k + 1 \leq 2^k + 1 \leq 2^k + k \leq 2^k + 2^k \leq 2^{k+1},$$

which closes the induction. □

See

- `https://www.mathsisfun.com/algebra/mathematical-induction.html`

for more examples. Use induction for the following problems.

**Exercise 2.1.**  Prove that for all $n \geq 1$,

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \sum_{i=1}^{n} i^2 = \frac{n(2n+1)(n+1)}{6}.$$

**Exercise 2.2.**  Prove that for all $n \geq 1$,

$$1 + 2 + 2^2 + \cdots + 2^n = 2^{n+1} - 1.$$

**Exercise 2.3.**  Prove that for all $n \geq 1$, $n^3 - n$ is divisible by 3.

**Exercise 2.4.**  Prove that if $x > -1$ where $x \in \mathbb{R}$, then

$$(1 + x)^n \geq 1 + nx$$

for any $n \geq 1$ where $n \in \mathbb{N}$.

**Exercise 2.5.**  Prove that if $n \geq 4$, then $n^2 \leq 2^n$.

**Exercise 2.6.**  Prove that if $A$ is a non-empty subset of $\mathbb{N}$, then $A$ has a least element, i.e. there is some $n_0 \in A$ such that for all $n \in A$ we have $n_0 \leq n$.

# 3 Strings

**Definition 3.1.** An alphabet is a finite set of symbols, usually denoted $\Sigma$.

**Definition 3.2.** A string $w = a_1 a_2 \ldots a_n$ over an alphabet $\Sigma$ is a finite sequence of symbols where each $a_i \in \Sigma$. The set of all strings over $\Sigma$ is denoted $\Sigma^*$.

If you have seen recursion in computer science, we may instead define them this way

**Definition 3.3.** A string $w$ over an alphabet $\Sigma$ is either

- the empty string, denoted $\epsilon$, or
- $ax$ where $a \in \Sigma$ and $x \in \Sigma^*$ is a string.

**Definition 3.4.** The length of a word $w$ is denoted by $|w|$ and is the number of symbols in $w$. Inductively,

- if $w = \epsilon$, then $|w| = 0$, and
- if $w = ax$ for $a \in \Sigma, x \in \Sigma*$, then $|w| = 1 + |x|$.

**Definition 3.5.** The concatenation of two strings $u = a_1 a_2 \dots a_m$ and $v = b_1 b_2 \dots b_n$ is $uv = a_1 \dots a_m b_1 \dots b_n$. Inductively, we have

- If $u = \epsilon$, then $uv = \epsilon v = v$, or
- If $u = ax$ for $a \in \Sigma, x \in \Sigma^*$, then $uv = a(x \cdot v)$.

**Remark 3.6.** *String concatenation is associative but not commutative.*

**Definition 3.7.** Let $\Sigma$ and $\Gamma$ be two alphabets. A string homomorphism is a function $h : \Sigma^* \to \Gamma^*$ such that for all $u, v \in \Sigma^*$, $h(uv) = h(u)h(v)$.

**Definition 3.8.** The $k$th power of a string $w$ is $w^k = \overbrace{w \dots w}^{k \text{ times}}$. Inductively

- $w^0 = \epsilon$, and
- $w^k = w^{k-1}w$.

**Definition 3.9.** If $w = uxv$ is a string, we say $u$ is a prefix, $v$ is a suffix, and $x$ is a substring of $w$.

**Question 3.10.** *What are the prefixes, suffices and substrings of "cat"?*

**Definition 3.11.** The reversal of a string $w$ is defined by

- if $w = \epsilon$, then $\epsilon^R = \epsilon$, and
- if $w = ax$ for a symbol $a \in \Sigma$ and a string $x \in \Sigma^*$, then $(ax)^R = x^R a$.

## 3.1 Languages

**Definition 3.12.** A language $L$ over $\Sigma$ is a subset $L \subset \Sigma^*$.

Some examples

- $\{w \in \{a,b\}^* : |w|_a \geq |w|_b\}$
- $\{w \in \Sigma^* : w = w^R\}$,
- $\{w \in \{0,1\} : w$ is the base-2 representation of a prime number$\}$
- $\{w \in \{A,C,G,T\}^* : ACC$ is not a substring of $w\}$.

**Definition 3.13.** The complement of $L$ is $\overline{L} = \Sigma^* \setminus L$.

**Definition 3.14.** Let $L_1, L_2$ be two languages. We define the concatenation by

$$L_1 L_2 = \{uv : u \in L_1, v \in L_2\}.$$

Just like string concatenation, language concatenation is not commutative.

**Definition 3.15.** The $k$th power of a language is

- $L^0 = \{\epsilon\}$, and
- $L^k = L^{k-1}L$.

**Definition 3.16.** The Kleene star of $L$ is defined by

$$L^* = \bigcup_{i \geq 0} L^i.$$

We define the positive Kleene closure of $L$ by $L^+ = LL^*$.

**Exercise 3.1.**  Let $w \in \Sigma^*$. Show that $(w^R)^i = (w^i)^R$ for all $i \in \mathbb{N}$.

**Exercise 3.2.**  Let $x, y \in \Sigma^*$. Show that $|xy| = |x| + |y|$.

**Exercise 3.3.**  Let $A, B, C \subset \Sigma^*$ be languages. Prove that $(A \cup B)C = AC \cup BC$.

**Exercise 3.4.**  Let $\Sigma = \{a,b\}$ and $Z_1 = \{w \in \Sigma^* : |w|_a = |w|_b\}$. Prove that

$$\mathrm{Pre}(Z_1) = \{a,b\}^*.$$

**Exercise 3.5.**  Let $u, v, x, y \in \Sigma^*$ and suppose that $uv = xy$. Prove that if $|u| \geq |x|$, then there exists $t \in \Sigma^*$ such that $u = xt$ and $y = tv$.

**Exercise 3.6.**  Let $L \subset \Sigma^*$ be a language. Show that $L^2 \subset L$ if and only if $L = L^+$.

**Exercise 3.7.** 

(a) Given a homomorphism $h : \Sigma^* \to \Sigma^*$, natural number $n \in \mathbb{N}$ and $w \in \Sigma^*$, give a formal inductive definition for $h^n(w)$.

(b) Prove that for all homomorphisms $h : \Sigma^* \to \Delta^*$ and natural numbers $n$ such that $h^n(uv) = h^n(u)h^n(v)$ for all strings $u, v \in \Sigma^*$.

(c) Let $\Sigma = \{0, 1\}$. Consider the sequence of strings $\{X_n\}_{n \geq 0}$, defined by

- $X_0 = 0$,
- $X_1 = 01$, and
- $X_n = X_{n-1}X_{n-2}$ for $n \geq 2$.

Let $\varphi$ be the homomorphism satisfying $\varphi(0) = 01, \varphi(1) = 0$. Prove that $X_n = \varphi^n(0)$ for all $n \in \mathbb{N}$.